Avaliação do piloto — agentes de IA no processo em cascata do Math Scape

Equipe: Eric Costa Ciciliotti, Roberta Telles, Sarah Mota e Yuri Martinelli · Data: 06/11/2026 · Commit avaliado: `b31c756` · Situação: **rascunho para revisão da equipe**

Documento da etapa "Avaliação, governança e institucionalização". O piloto comparado aqui é o Math Scape construído em 8 fases com os cinco agentes do trabalho `agentes-ia-processo-cascata.pdf`, cada um com o seu harness. A norma que sai desta avaliação está em norma-processo-ia.md.

Resumo

- **O produto bateu a linha de base do legado onde foi medido.** O legado tinha respostas no navegador, 0 testes e 1 fase fixa. O sistema novo esconde as respostas, tem 667 testes de unidade e integração e 96 de sistema, e suporta fases ilimitadas. Também cobre 95,5% do schema e 100% do motor. **O critério de usabilidade não foi medido**: não houve sessão com usuários (RNF-08 e SUS em branco).
- **O shift left aconteceu.** O agente 5 achou 42 lacunas de projeto antes da primeira linha de código, todas resolvidas antes da fase 4. Uma delas teria custado trabalho do criador (LAC-003). Depois do código, escaparam 10 defeitos e 1 diferença de requisito.
- **Os agentes 3 e 4 não pegaram os defeitos que escaparam.** O agente 4 ficou verde no 1º ciclo em 46 de 47 implementações, sem nenhuma reversão. Mesmo assim, os 10 defeitos estavam fora do que os testes de unidade olham: a composição das telas, o layout real, o seed e o proxy.
- **Três modos de falha explicam quase tudo o que escapou:**
  - achado de agente que não virou pendência (4 defeitos);
  - decisão humana delegada à IA (2 defeitos);
  - "verde que esconde a composição" (4 defeitos).

  Um mesmo defeito pode cair em mais de um modo.
- **A sessão principal custou mais que os agentes.** Foram 68% dos 261 milhões de tokens de entrada medidos. Os agentes com harness gastaram 32%.
- **O agente 2 (manutenção) não foi exercitado.** Nenhum chamado foi aberto, então não há dado para comparar.
- **Decisão proposta:** institucionalizar os agentes 1, 3, 4 e 5 com as mudanças da seção 7. O agente 2 fica em modo sombra até ter dados.

1. Escopo, linha de base e fontes

O que foi o piloto

Período: 31/10 a 06/11/2026. O plano é de 27/10 e o último commit é de 06/11. Foram 83 commits, 76 deles em 05 e 06/11. Ferramenta: Claude Code com o modelo Claude Opus 5.5, em todas as fases. Os agentes 1, 3, 4 e 5 rodaram como subagentes com harness. As fases 5 a 8 rodaram na sessão principal, sem harness.

As duas linhas de base

Linha de base	O que representa	De onde vem	Limite
Produto: o legado	Math_Scape_Telas, HTML/CSS/JS	Coluna "Legado" da seção 8 do plano	Medida antes do projeto; é confiável
Processo: cascata sem agentes	O mesmo processo de 8 fases, com a verificação concentrada no fim	Seção 1 do trabalho dos agentes e plano de 27/10	**Não houve grupo de controle.** Os números do processo original são de referência, não medidos nesta equipe

Por isso, as comparações de processo abaixo mostram quando cada defeito foi achado e quanto custou achá-lo. Elas não afirmam que o cascata puro teria levado 8 meses neste projeto. Este MVP é pequeno, e o próprio plano já pedia critérios de aceitação na fase 1.

Fontes

- Relatórios do harness: 97 arquivos em `harness/relatorios/` (1 do agente 1, 15 do agente 5, 15 do agente 3 e 65 do agente 4).
- Log de decisões: `harness/log_decisoes.jsonl`, com 1.373 linhas de 05/11 23:17 a 06/11 20:03. **Está incompleto**: o log não é versionado e as rodadas anteriores ficaram em outra máquina (seção 3, F-08).
- Pendências: 25, em `harness/pendencias/`. Lacunas: 53, em `harness/lacunas/`.
- Logs de IA: `docs/ia-log/fase-1..8.md` e a seção 11 do Documento de Requisitos v1.1.
- Testes, defeitos e aceitação: `docs/testes/` e `docs/aceitacao/`. Operação: `docs/operacao/`.

2. Resultados contra a linha de base

2.1 Produto: métricas da seção 8 do plano

Métrica	Legado	Depois (medido)	Fonte
Como se cria uma fase nova	Escrevendo HTML, CSS e JS	Editor visual, sem código. Fluxo criar → publicar → jogar concluído em Chromium, Edge e Firefox	E2E `fluxo-completo`
Tempo para criar uma fase simples	não medido	**Não medido.** As sessões com usuários da fase 7 não foram feitas (checklist "em preparação")	`checklist-aceitacao.md`
Respostas e senha visíveis no navegador	Sim	**Não.** Tráfego e JavaScript inspecionados no jogo e no modo testar	RNF-04, fase 6
Onde fica a lógica de cada objeto	`onclick` com função própria	Documento JSON validado por um schema único e interpretado pelo motor	RNF-09
Testes automatizados	0	667 de unidade e integração + 96 de sistema (+1 só de evidência) + 60 do próprio harness	relatório da fase 6
Cobertura do núcleo	0%	Schema 95,5% das linhas e 92,2% dos ramos · motor 100% e 100%	`cobertura.sh`
Fases disponíveis	1, fixa no código	Ilimitadas. **Ressalva:** num banco novo a Escola não aparece, porque o seed não existe (DEF-09, alta, aberto)	DEF-09
Nota SUS	não medida	**Não medida**	fase 7

Critérios de sucesso do plano

Critério	Situação
Fase original com o mesmo comportamento do legado	**Atendido no comportamento.** 41 verificações de equivalência iguais e 4 falas diferentes (DIF-01, decisão da Roberta). Faltam o quadro desenhado (DEF-06) e a carga em produção (DEF-09)
Pessoa sem programação publica em menos de 30 min	**Não verificado**
Nenhuma resposta ou senha no navegador	**Atendido**
Uso de IA registrado por fase	**Parcial.** Fases 1 a 6 medidas; fases 7 e 8 sem tokens

Defeitos abertos no commit avaliado: **DEF-09 e DEF-10 (altos)**, DEF-06 (médio), DEF-07 e DEF-08 (baixos). Pela regra do próprio checklist, a aceitação seria "não aceito" até os dois altos serem corrigidos.

2.2 Processo: as métricas de cada agente

As métricas são as que o trabalho define para cada agente (seção 3 do PDF).

Agente	Métrica definida	Referência sem agente	Piloto	Leitura
1 · requisitos	% de requisitos com critério verificável	Critérios escritos na fase 7	Rascunho: 25 de 29 (86%) com critério. Baseline: 28 de 28 (100%), depois do gate humano	Atendido
	Densidade de ambiguidade	—	4 itens não fechados (2 ambíguos, 2 em conflito) e 6 pendências numa especificação de cerca de 2,5 páginas (1.253 palavras)	Achou o conflito do RF-18 × 4 tipos de objeto, que não estava no documento
	Defeitos de requisito vazados	Todos, descobertos na aceitação	**3** dos 11 achados posteriores: DIF-01, DEF-06 (RF-18 sem escopo visual) e DEF-04 (critério do RF-03 não fala de layout)	Vazaram critérios de **aparência** e de **ergonomia**, não de regra
2 · manutenção	Tempo até diagnóstico, duplicatas capturadas, reclassificação	—	**Sem dado.** 0 chamados em `harness/chamados/`	DEF-09 e DEF-10 apareceram antes da operação e não passaram pelo agente 2
3 · testes	Cobertura de borda por requisito	Borda é o que se corta sob prazo	26 de 28 requisitos com feliz, borda e negativo. Ficaram fora o RNF-03 (medição) e o RNF-08 (usuários). 15 rodadas, todas `sucesso`, 0 devolvidas ao agente 1	Atendido
	Razão negativo/feliz	—	Mediana 1,09. **10 de 26 requisitos abaixo de 1**; o menor é o RF-18 (0,46)	Equilibrado na Api e no schema, fraco nas telas
	Defeitos pegos na fase 5 que antes só apareciam na 7	—	Os testes nasceram vermelhos e guiaram o código. **Nenhum dos 10 defeitos posteriores foi pego pela suíte do agente 3**	Ver F-02 e F-04
4 · codificação	Ciclos editar-testar até o verde	—	65 execuções: 46 verdes no 1º ciclo, 1 no 2º, 18 confirmações com 0 ciclos e 1 escalada (teste defeituoso, PEND-013). O teto de 5 nunca foi atingido	Convergência alta demais para ser só mérito (F-04)
	Taxa de regressão	—	**0 reversões** pela P3	Atendido
	% do diff fora do escopo	—	**0.** No log disponível não houve nenhuma tentativa de editar fora do escopo. Os 82 bloqueios P1 do agente 4 foram comandos de shell fora da lista	Atendido, com atrito (F-06)
5 · projeto	% de requisitos rastreados até componente	—	28 de 28 em todas as 15 rodadas	Atendido
	RNF crítico sem decisão	—	0	Atendido
	Tempo entre mudança de requisito e impacto	Meses	Baseline `2026-11-03.1` verificado na mesma sessão (rodada 3, 03/11). A rodada pós-fase 6 achou 11 lacunas em 3,5 min	Atendido
	Lacunas	—	53 no total: 46 resolvidas e 7 em rascunho (LAC-047 a 053). Bloqueantes: 3 na rodada 1 da fase 2, 0 a partir da rodada 3	Ver F-05 sobre o custo de convergência

2.3 Latência do defeito: onde nasceu e onde foi achado

Antes do código (shift left). O agente 5 registrou 26 lacunas na fase 2 e 16 na fase 3, todas resolvidas antes da fase 4. Entre elas:

- **LAC-003:** o salvamento automático mandaria estados que o próprio schema recusa, e o criador perderia trabalho;
- **LAC-015 e LAC-021:** um limite por IP bloquearia a turma inteira no laboratório, e um limite por partida não seguraria ninguém;
- **LAC-028:** o banco da suíte ignoraria os `CHECK` do SQL;
- **LAC-031:** o legado não estava no workspace, e sem ele o agente 4 teria de inventar as perguntas.

Sem o agente, essas lacunas apareceriam como defeito na fase 6 ou 7.

Depois do código. Os defeitos e a diferença registrados nas fases 5 a 8:

Item	Prioridade	Nasceu em	Achado em	Algum agente já tinha apontado?	Por que escapou
DEF-01	Alta	3, contrato do Editor sem controles	5, leitura de código	não	Store testado e verde; nenhum teste monta a tela inteira
DEF-02	Alta	4, teste escrito para clique, projeto pede arrastar	5	não	Teste e código concordaram no comportamento errado
DEF-03	Média	4, `uuid` × `asset:uuid`	6, E2E	não	Cliente falso nos testes de tela
DEF-04	Média	4, **PEND-019 respondida pela IA**: palco do tamanho da janela	6	não	Convenção de teste virou comportamento de produto
DEF-05	Alta	4, palco do Player do tamanho da janela	6	**sim**, agente 4 no RF-16: "pode transbordar na vertical"	Ficou como nota no log, sem pendência
DEF-06	Média	1, RF-18 sem escopo visual	6	sim, agente 4 no RF-18: "equivalência visual para a fase 6"	Requisito silencioso
DEF-07	Baixa	3, sem contrato comum de textos	6	**sim**, log da fase 4: "textos fixos no `ModoTestar`"	Nota sem pendência
DEF-08	Baixa	4, **PEND-014 respondida pela IA**: adiar o `browserslist`	5	sim, na própria PEND-014	Adiamento sem dono nem prazo
DEF-09	Alta	3, seed especificado em `modulos-banco.md`	7	não	Nenhum teste pedia o seed, e o agente 4 só faz o que os testes pedem
DEF-10	Alta	4, sem `trustProxy`	8, pilha de produção	**sim**, agente 4 no RF-01: "atrás do proxy, todos podem dividir o IP. Decidir na implantação"	Nota sem pendência
DIF-01	—	1, RF-18 × 4 tipos	6	parcial (PEND-001)	Decisão de requisito ainda aberta

Números da tabela:

- **5 de 11** foram apontados por algum agente antes de virarem defeito. Os agentes viram o problema, mas o processo não fez a informação andar.
- **2 de 11** nascem de decisões que a IA tomou no lugar de uma pessoa.
- **4 de 11** só aparecem com o sistema montado (DEF-01, 03, 09 e 10). Essa é a camada que nenhum agente com harness testou.

2.4 Custo

Tokens medidos nas transcrições. A entrada inclui o contexto relido a cada passo (cache), por isso é muito maior que o texto das mensagens.

Fase	Responsável	Entrada	Saída	Origem
1 · Requisitos	Roberta (31/10 a 01/11)	~2.525.000	~91.000	parcialmente estimado
1 · Requisitos no harness	Eric (03/11)	12.147.105	149.938	medido
2 · Projeto do sistema	Eric	26.237.070	237.106	medido
3 · Projeto do programa	Eric	70.876.142	325.979	medido
4 · Codificação	Eric	140.815.934	1.427.644	medido
5 · Unidade e integração	Sarah	2.917.029	51.652	medido
6 · Sistema	Sarah	8.061.953	103.081	medido
7 · Aceitação	Yuri	—	—	**não preenchido**
8 · Operação	Yuri	—	—	**não preenchido**
Total		**~263,6 milhões**	**~2,39 milhões**	6 de 8 fases

Quem gastou (fases 1 a 6, só o medido):

Ator	Entrada	%
Sessão principal: orquestra, revisa, decide e commita	176,7 M	**67,7%**
Agente 4 (implementar)	37,2 M	14,2%
Agente 3	30,1 M	11,5%
Agente 5	14,8 M	5,7%
Agente 1 (inclui a rodada descartada)	0,75 M	0,3%
Agente 4 (só confirmar, 0 ciclos)	1,4 M	0,5%

Tempo de execução dos agentes (início e fim nos relatórios):

Agente	Execuções	Duração mediana	Soma das durações
Agente 4	65	2,6 min	~3 h
Agente 3	15	7,3 min	~1 h 46 min
Agente 5	15	3,2 min	49 min
Agente 1	1	6,5 min	6,5 min

O gargalo foi a supervisão. O plano não converte tokens em dinheiro, porque a maior parte da entrada é leitura de cache, com outro preço. A conversão fica como ação (seção 7).

3. Modos de falha

Cada modo vem com a evidência e o controle que a norma passa a exigir.

#	Modo de falha	Evidência	Efeito	Controle na norma
F-01	**Harness desligado em silêncio.** Na extensão do VS Code, os hooks do frontmatter não rodam, e o agente trabalha sem gate	3 incidentes: a rodada da Roberta em 31/10 (hooks no PowerShell, sem log), a 1ª rodada do agente 1 em 03/11 e a 1ª rastreabilidade de 06/11. Na de 03/11, **o modelo relatou "38 de 40 iterações" como se o harness contasse**	Métrica inventada com aparência de controle. Só a ausência de log revelou	N-03

#	Modo de falha	Evidência	Efeito	Controle na norma
F-02	**Verde que esconde a composição**	DEF-01, 03, 09 e 10. Os 607 testes estavam verdes quando a Sarah achou DEF-01 lendo o código	O agente 4 converge para o que a suíte pede. O que não tem teste, como `main.tsx`, o seed ou o proxy, não existe para ele	N-14, N-15
F-03	**Achado que não vira pendência**	DEF-05, 06, 07 e 10 estavam nos relatórios e no log da fase 4 como "achados"	O defeito sobe 2 a 4 fases. A informação existia	N-06
F-04	**Erro correlacionado entre teste e código**	DEF-02: o teste foi escrito para clique e o código para clique; o projeto pedia arrastar. 46 de 47 verdes no 1º ciclo	A P2 do agente 4 separa os atores, mas não o modelo. O mesmo modelo, lendo o mesmo contrato, erra igual dos dois lados. A P3 do agente 3 só aproxima "não inventa requisito"	N-16
F-05	**Convergência cara do agente 5**	Fase 3: 8 rodadas, com 6, 1, 2, 1, 2, 3, 1 e 0 lacunas novas. A 1ª estourou o teto de 80 iterações. A sessão principal da fase gastou 61 M de tokens para resolver as lacunas	Custo alto para lacunas cada vez menores (a rodada 7 achou "só texto")	N-19
F-06	**Atrito do gate (falsos positivos)**	89 comandos de shell barrados pela P1 no log disponível, 46 deles por prefixar `cd "…" &&`. A P2 barrou uma refatoração de helper dentro de `waitFor`, sem afrouxar a asserção. A P4 marcou "senha" do jogo como credencial	Iterações gastas. Nenhum contorno do gate foi observado	N-22
F-07	**Estado perdido na retomada**	Limite de uso no RF-09 (madrugada de 06/11). O `SubagentStart` zerou a tarefa, e o gate barrou 3 edições (TAREFA)	O agente reiniciou certo, mas por sorte do prompt	N-23
F-08	**Trilha de auditoria fragmentada**	`log_decisoes.jsonl` é ignorado pelo git. As linhas de 03/11 a 05/11 ficaram em outra máquina, e os relatórios não registram o modelo	A auditoria "linha a linha" do desenho não é reproduzível depois	N-08, N-09
F-09	**Decisão humana delegada à IA**	PEND-013 a 021 (9 de 25) com `respondida_por: "Claude (sessão principal), com decisão delegada pelo Eric"`. O contrato das telas do Editor foi decidido pela IA "como arquiteta". As 17 ADRs seguem "proposta" (LAC-049, LAC-052)	2 defeitos (DEF-04, DEF-08). Decisões sem dono ratificado	N-04, N-05
F-10	**Trabalho de teste fora do harness**	Fases 5 e 6 na sessão principal. Os 97 casos E2E não passaram pelo gate (P2, P3 e P4). Corridas do teste pareceram defeitos, e uma execução da IA apagou resultados	A fase que mais achou defeito foi a menos controlada	N-17
F-11	**Medição incompleta**	Fases 7 e 8 sem tokens. Aceitação sem participantes	Não dá para fechar a comparação de custo nem o RNF-08	N-10

4. Onde a delegação não se justificou

A regra para julgar vem do próprio desenho: **delegar quando o erro é recuperável e barato. Quando a decisão é irreversível, auditável ou de outra pessoa, ela não é do modelo.**

Caso	Por que não se justificou	Evidência	O que fazer
Responder pendências (PEND-013 a 021)	Pendência é, por definição, o que o agente não pode decidir. Delegar a resposta à IA da sessão principal anula o gate humano por outro caminho	2 defeitos. A PEND-019 parecia convenção de teste e fixou o tamanho do palco no produto	Proibido (N-04). A IA pode redigir a recomendação; a pessoa responde
Confirmar requisito sem editar (18 execuções do agente 4)	Verificação determinística (suíte verde e diff vazio) feita por um modelo. Barato (1,4 M), mas está na camada errada	18 relatórios com 0 ciclos	Ferramenta sem modelo (N-18)
Rodadas finais do agente 5 na fase 3 (4 a 8)	Lacunas não bloqueantes cada vez menores. Cada rodada exigia outra rodada de correção na sessão principal	2,1 M de tokens do agente para 7 lacunas não bloqueantes, mais a correção	Regra de parada por retorno (N-19)
Sessões sem a pessoa (05/11 à noite, "Eric fora")	A IA orquestrou 34 entregas, revisou o trabalho de outros agentes e decidiu pendências. Era IA revisando IA, sem o humano do "revisão humana sempre"	Entregas 26 a 59; LAC-049	Revisão humana antes do commit (N-05)
Rodar agente sem harness	Sem gate, o agente é só um prompt	F-01: 3 incidentes, ~0,4 M de tokens descartados no mais caro	Execução sem log é descartada (N-03)
Medir o RNF-08 e a SUS	Não dá para delegar: precisa de pessoas reais	Checklist em branco	A IA prepara o kit; a sessão é humana (N-13)

Onde a delegação se justificou, para equilíbrio:

- **Agente 5 nas rodadas 1 a 3 da fase 2:** 2,3 M de tokens para achar LAC-003, LAC-015 e LAC-021, que só apareceriam na aceitação.
- **Par agente 3 → agente 4:** 0 regressões, rastreio 28 de 28 e cada requisito verde num commit auditável.
- **IA como investigadora nas fases 6 a 8:** achou DEF-09 e DEF-10 antes de produção e provou o DEF-10 com dois clientes de IPs diferentes.
- **Agente 2 não ter sido usado:** sem chamados, ele não tinha o que fazer. O ganho dele é por acumulação.

5. Riscos

5.1 Segurança

Risco	Situação no piloto	Controle existente	Lacuna
Agente com shell na máquina de desenvolvimento	Lista de comandos permitidos por agente (P1); testes em contêiner `--network none`	Gate e `settings.json`	**A sessão principal não tem gate.** Ela abriu o Docker Desktop, construiu imagens e mexeu na infra, inclusive sem o Eric presente
Segredo no código	Padrões de segredo na P4 do agente 4; `.env` ignorado	Gate	O E2E e a infra foram escritos fora do gate
Dado pessoal no contexto	Log mascarado antes do agente 2 (`mascaramento.js`); leitura de `harness/logs/brutos/` negada	Harness e `settings.json`	Os dados dos participantes da fase 7 devem ficar fora de qualquer prompt (códigos P1… P6)
Injeção de instrução por dados	Legado, issues e logs entram no contexto como texto	Agente 2 sem acesso a produção, escrita só de diagnóstico	Chamados vêm de issues públicas: o texto do usuário é dado, nunca instrução
Cadeia de dependências	Só o arquiteto mexe em `package.json`	P1 do agente 4	O pnpm resolveu o Prisma para uma versão RC e foi corrigido à mão. O Playwright entrou num manifesto à parte (`tests/e2e/package.json`), fora da regra; vale ratificar

Risco	Situação no piloto	Controle existente	Lacuna
Segurança do produto	RNF-04 aprovado; login com tempo constante; limite de tentativas	Testes e E2E	**DEF-10 aberto:** negação de serviço atrás do proxy

5.2 Propriedade intelectual

- **Autoria do código.** 81 de 83 commits trazem `Co-Authored-By: Claude`. A atribuição está registrada, e isso é bom para a disciplina. A titularidade do que o modelo gera depende dos termos do fornecedor vigentes no contrato da conta. Isso não foi verificado e fica como ação.
- **Sem licença no repositório.** Não há arquivo `LICENSE`. Sem licença, ninguém de fora pode reutilizar o código, e a equipe não definiu como dividir a autoria entre os quatro.
- **Imagens do legado.** Os sprites de `packages/schema/biblioteca/sprites/` vieram do Math_Scape_Telas, e a origem delas não está documentada. Se alguma veio da internet sem licença, o sistema publicado a redistribui.
- **Uso acadêmico.** O plano exige o registro do uso de IA. Os logs das fases 1 a 6 cumprem isso. Os das fases 7 e 8 estão incompletos.
- **Material do professor e da equipe enviado ao fornecedor.** O repositório inteiro, inclusive os PDFs, entrou no contexto do modelo. Para trabalho acadêmico isso é aceitável. Para código de cliente, exigiria autorização.

5.3 Dependência de fornecedor

Peça	Portável?	Por quê
Políticas (`.claude/hooks/agentes/*.js`, `.claude/lib/*.js`, cerca de 1.800 linhas)	**Sim**	Node puro, testado sem modelo (`node --test`, 60 casos)
Artefatos (`harness/*.json`, baseline, matriz, pendências, lacunas)	**Sim**	JSON com esquema próprio
Ligação (frontmatter `hooks:` e `tools:`, eventos `PreToolUse`/`SubagentStop`/`SubagentStart`, nomes `Read`/`Edit`/`Bash`)	**Não**	Específica do Claude Code. Mesmo dentro do fornecedor, a superfície importa: a extensão do VS Code ignorou os hooks (F-01)
Modelo	**Não testado**	Um só modelo (Opus 5.5) em todas as fases, sem comparação. Os relatórios não registram o modelo
Disponibilidade	—	O limite de uso da conta parou um agente no meio (F-07)

A boa notícia do desenho é que **a garantia está no código, não no prompt**. Trocar de ferramenta exige reescrever a ligação, não as políticas. O custo da saída é pequeno e conhecido, mas só vale se for exercitado (N-26).

5.4 Efeitos sobre a formação da equipe

O que os dados mostram:

- **O trabalho se concentrou numa pessoa.** São 75 commits do Eric, 6 da Sarah, 2 do Yuri e 0 da Roberta (o baseline dela entrou pelo Eric). Na sessão de 05 e 06/11, a IA conduziu sozinha 34 entregas (26 a 59).
- **Os papéis viraram revisão.** Na fase 5, a IA propôs as prioridades dos defeitos, e a Sarah confirmou. Na fase 6, a IA escreveu a suíte Playwright. Nas fases 7 e 8, a IA montou o kit, a infra e o manual, e o Yuri corrigiu.
- **A revisão funcionou.** Todo log tem "onde errou" com correções humanas concretas: a rota inexistente no manual, as setas do teclado no Player, o corte de 8 MB no proxy e o falso defeito das falas do legado.

Riscos:

- **Assinar o que não se produziu.** A assinatura deixa de provar domínio. Quem assina o plano de testes precisa saber explicar por que um caso existe.
- **Perder a prática do ofício.** Sem escrever testes, triar defeitos ou configurar uma implantação à mão, a equipe não forma o critério que precisa para revisar a IA. Esse critério é justamente o que pegou os erros acima.
- **Aprender a operar o harness em vez do processo.** Só o Eric conhece o código do harness: é um ponto único de falha.

Ganho: a equipe aprendeu a ler pendências, lacunas e relatórios como artefatos de engenharia, e a separar julgamento de garantia. Esse ganho só se mantém se cada pessoa continuar fazendo uma parte do trabalho à mão (N-27 a N-30).

6. Respostas às perguntas da etapa

- **O piloto foi melhor que a linha de base?** No produto, sim, em tudo que foi medido. No processo, a latência dos defeitos de projeto caiu: 42 lacunas foram resolvidas antes do código. Os defeitos de composição, porém, continuaram aparecendo tarde, como no cascata puro.
- **Onde falhou?** Na passagem de informação, não na detecção: achados sem pendência e decisões sem dono. E na camada que nenhum agente cobre, a do sistema montado.
- **Vale institucionalizar?** Sim para os agentes 1, 3, 4 e 5, com a norma. O agente 2 ainda não, por falta de dado.

7. Decisão de institucionalização e plano de ação

Agente	Decisão	Mudança obrigatória
1 · requisitos	**Adotar**	Critério de aparência e ergonomia nos requisitos de tela; ajustar a P4 para "senha do jogo" (glossário)
3 · testes	**Adotar, ampliado**	Camada de composição e E2E sob o harness (N-15, N-17); revisão por amostra contra o critério (N-16)
4 · codificação	**Adotar com restrições**	Achado vira pendência (N-06); confirmação passa a ser ferramenta (N-18)
5 · projeto	**Adotar**	Regra de parada por retorno (N-19); rodar depois de cada fase de teste
2 · manutenção	**Modo sombra**	Registrar DEF-09 e DEF-10 como chamados retroativos e comparar o diagnóstico do agente com o humano nos 10 primeiros chamados

#	Ação	Dono proposto	Quando
1	Corrigir DEF-09 e DEF-10, com o caso de teste antes	Eric (Sarah escreve o caso)	antes da aceitação
2	Ratificar ou substituir as decisões da LAC-049 e o status das 17 ADRs (LAC-052)	Eric	antes da aceitação
3	Fazer as sessões da fase 7 (RNF-08 e SUS) e preencher a seção 2.1	Yuri	próxima aula
4	Preencher os tokens das fases 7 e 8 e converter o total em custo pelo painel da conta	Yuri e Eric	com a ação 3
5	Ferramenta `confirmar-requisito` sem modelo; ajuste da P1 (aceitar o `cd` para a raiz do projeto); `ciclo.js` que preserva a tarefa na retomada	Dono do harness	próximo ciclo
6	Arquivar o `log_decisoes.jsonl` mascarado ao fim de cada fase e registrar o modelo nos relatórios	Dono do harness	próximo ciclo
7	`LICENSE`, origem dos sprites e conferência dos termos do fornecedor	Equipe	antes de publicar o link
8	Decidir o DIF-01	Roberta	antes da aceitação

8. Limitações desta avaliação

- **Não houve grupo de controle**, então a comparação de processo é contra a referência do desenho, não contra uma equipe sem agentes.
- **O mesmo modelo construiu o sistema e redigiu esta avaliação**, a partir dos artefatos. Os números vêm dos relatórios, do log e dos documentos; os julgamentos precisam de revisão da equipe antes da assinatura.
- **O log de decisões cobre só 05/11 23:17 a 06/11 20:03.** As contagens de bloqueio anteriores vêm dos relatórios.
- **Projeto pequeno, 7 dias de trabalho, 4 pessoas.** Os números de custo não se transferem para projetos maiores sem nova medição.

Revisado por: _____ · Data: ___/___/2026

Norma do processo de software com agentes de IA — Math Scape

Código: NP-01 · Versão: 1.0 (proposta) · Data: 06/11/2026 · Origem: avaliação do piloto · Situação: **aguarda aprovação da equipe**

1. Objetivo e alcance

Esta norma diz como a equipe usa agentes de IA nas 8 fases do processo em cascata do Math Scape: quem decide, o que os agentes podem fazer, o que se registra e o que fecha uma fase.

Ela vale para todo trabalho com IA no repositório, com ou sem harness: subagentes, sessão principal do Claude Code, chat e qualquer outra ferramenta.

Ela **não substitui** as regras que já estão em `CLAUDE.md` e `.claude/rules/` (fronteira modelo × harness, rastreabilidade, testes, commits). Ela as completa com o que o piloto mostrou que faltava. Em caso de conflito, vale a regra mais restritiva até a equipe decidir.

Linguagem: **deve** é obrigatório; **não deve** é proibido; **pode** é permitido.

2. Referências

- `Math Scape Plano de Reengenharia com IA.docx` (27/10/2026): fases, responsáveis e regras de uso de IA.
- `agentes-ia-processo-cascata.pdf`: os cinco agentes e o harness mínimo.
- `CLAUDE.md` e `.claude/rules/*.md`.
- Documento de Requisitos v1.1 e `harness/baseline/requisitos.json`.

3. Definições

Termo	Significado
Agente	Um dos cinco subagentes em `.claude/agents/`, executado pela skill correspondente
Harness	Código determinístico em volta do agente: gate, estado, orçamento, parada e log
Execução válida	Execução de agente com relatório em `harness/relatorios/` **e** linhas em `harness/log_decisoes.jsonl`
Sessão principal	Conversa com a IA fora dos subagentes. Não tem gate: quem garante é a pessoa
Pendência	Pergunta que um agente não pode decidir (`harness/pendencias/`)
Lacuna	Inconsistência entre requisito e projeto registrada pelo agente 5 (`harness/lacunas/`)
Achado	Qualquer observação de agente ou da IA que indique risco de comportamento, segurança, operação ou equivalência
Responsável da fase	Pessoa indicada no plano para a fase (Roberta, Eric, Sarah, Yuri)
Dono do harness	Pessoa que mantém `.claude/` e responde por mudanças nas políticas

4. Papéis

Atividade	Responsável da fase	Arquiteto (Eric)	Requisitos (Roberta)	Testes (Sarah)	Aceitação e operação (Yuri)	Dono do harness	IA
Executar o agente da fase	**R**	C	C	C	C	—	executa

Atividade	Responsável da fase	Arquiteto (Eric)	Requisitos (Roberta)	Testes (Sarah)	Aceitação e operação (Yuri)	Dono do harness	IA
Responder pendência de requisito	I	C	**R/A**	I	I	—	recomenda
Responder pendência ou lacuna de arquitetura	I	**R/A**	C	C	I	—	recomenda
Aceitar ADR	—	**R/A**	C	I	I	—	redige
Priorizar defeito	C	C	I	**R/A** (5-6)	**R/A** (7-8)	—	propõe
Assinar artefato da fase	**R/A**	—	—	—	—	—	nunca
Mudar política do harness	I	C	I	I	I	**R/A**	redige, com teste
Commit e push de fase	**R/A**	—	—	—	—	—	executa, com revisão

R = executa · A = aprova · C = é consultado · I = é informado.

A IA nunca aprova. Ela executa, redige, recomenda e propõe.

5. Regras gerais

N-01 · Uso. A IA é ferramenta de engenharia, não parte do produto. Quem cria as fases do jogo é sempre a pessoa (plano, seção 1).

N-02 · Fonte da verdade. O baseline de requisitos e o schema da fase mandam. Uma sugestão da IA que os contradiga é descartada ou vira pedido de mudança formal (pendência para a Roberta, `/promover-baseline`).

N-03 · Só execução válida conta.

- Os agentes devem rodar pelo **CLI do Claude Code no terminal**. Na extensão do VS Code, os hooks do frontmatter não rodam.
- Antes de aceitar o resultado, o responsável **deve** conferir que existe o relatório da execução e que o log ganhou linhas daquele agente.
- Execução sem relatório ou sem log é **descartada**: não entra em commit e não entra em métrica, mesmo que o agente diga que o harness contou. No piloto, o modelo relatou "38 de 40 iterações" sem harness ativo.

N-04 · Decisão é de pessoa.

- Pendência, lacuna, ADR e prioridade de defeito são respondidas por uma **pessoa identificada pelo nome**, no papel da seção 4.
- A IA **pode** escrever uma recomendação, com as opções e o efeito de cada uma. Ela **não deve** preencher `respondida_por` nem a resposta.
- "Decisão delegada à IA" **não é aceita**. Quem não pode decidir agora deixa a pendência aberta, e o trabalho que depende dela espera ou segue em ramo.
- Uma pergunta sobre como testar não pode fixar um comportamento do produto. Se a convenção de teste muda o que o usuário vê, a pendência deve dizer isso de forma explícita. No piloto, a PEND-019 fixou o palco do tamanho da janela e gerou o DEF-04.

N-05 · Revisão humana antes da `main`.

- Nada gerado pela IA entra na `main` sem a revisão do responsável da fase, que lê o diff e o relatório.
- Uma sessão de IA sem a pessoa presente **pode** produzir trabalho num ramo `ia/<fase>-<data>`, mas **não deve** commitar na `main` nem dar push nela.

- A revisão de uma entrega de agente feita pela sessão principal **não conta** como revisão humana.

N-06 · Achado vira registro.

- Todo achado de agente ou da sessão principal **deve** virar pendência (`/abrir-pendencia`) ou lacuna antes do fim da sessão em que apareceu. Nota em relatório, em comentário ou em log de IA não basta.
- No piloto, DEF-05, DEF-06, DEF-07 e DEF-10 estavam escritos como notas na fase 4 e só foram tratados 2 a 4 fases depois.
- Achado sobre produção, segurança ou proxy vai para o arquiteto com severidade.

N-07 · Escalar não é falhar.

- Pendência, recusa e reversão contam como sucesso parcial.
- Diante de um bloqueio do gate, o agente lê o motivo e ajusta a ação, ou escala.
- **Ninguém deve contornar o gate** por outra ferramenta, por outro arquivo ou pela sessão principal. Isso vale para pessoas também. Por exemplo, a sessão principal não deve escrever um teste que o agente 3 foi impedido de escrever.

6. Registro e medição

N-08 · Trilha de auditoria preservada.

- O `harness/log_decisoes.jsonl` continua fora do git.
- Ao fechar cada fase, o responsável **deve** arquivar uma cópia do log no armazenamento compartilhado da equipe, com o nome `log-fase-<N>-<data>.jsonl`. O log não deve conter dado pessoal (o mascaramento já roda antes da escrita).
- Ao trocar de máquina, o log da máquina anterior é arquivado antes.

N-09 · Relatório identifica o modelo. O relatório de cada execução **deve** registrar a ferramenta, a versão dela e o identificador do modelo. Até o harness gravar isso sozinho, o responsável anota no log de IA da fase.

N-10 · Tokens por sessão.

- Cada sessão registra ferramenta, modelo, tokens de entrada e saída e tarefa, no fim da sessão, em `docs/ia-log/fase-<N>.md`.
- A origem é "medido" (transcrição ou painel) ou "estimado" (tokenizer).
- **Fase sem total de tokens não fecha.**
- Ao fechar o projeto, o total é convertido em custo pelo painel de uso da conta.

N-11 · Métricas obrigatórias no fechamento da fase.

Agente	Métrica	Fonte
1	% de requisitos com critério verificável; itens não fechados; defeitos de requisito vazados (preenchido nas fases 5 a 8)	relatório e relatórios de defeito
2	tempo até diagnóstico; duplicatas pela P5; reclassificados como desvio de requisito; acerto contra a triagem humana	`harness/diagnosticos/`
3	requisitos com feliz, borda e negativo; razão negativo/feliz; devolvidos ao agente 1	relatório
4	ciclos até o verde; reversões; diff fora do escopo; achados abertos como pendência	relatório e log
5	% rastreado; RNF crítico sem decisão; lacunas por rodada e bloqueantes	relatório
Todos	execuções descartadas pela N-03; bloqueios por política, separando falso positivo de bloqueio legítimo	log

7. Regras por fase

N-12 · Entrada, saída e quem fecha.

Fase	Uso de IA	Critério de saída (além do da `.claude/rules/commits.md`)	Assina
1 · Requisitos	Agente 1; chat para engenharia reversa	Todo requisito de tela com critério de aparência e ergonomia, ou pendência explícita dizendo que não há	Roberta
2 · Projeto do sistema	Agente 5	ADRs que sustentam RNF crítico com status **aceita** (não "proposta")	Eric
3 · Projeto do programa	Agente 5; sessão principal para contratos	Todo item especificado (seed, configuração de proxy, script) com caso de teste previsto ou lacuna (N-14)	Eric
4 · Codificação	Agentes 3 → 4, um requisito por vez	Suíte verde; nenhum achado sem pendência (N-06); teste de composição de cada app (N-15)	Eric
5 · Unidade e integração	Agente 3 para casos novos; IA para cobertura e leitura de código	Cobertura do núcleo ≥ 80%; defeitos com prioridade dada pela Sarah	Sarah
6 · Sistema	E2E conforme a N-17	Sem defeito alto aberto; fluxo também atrás do proxy de produção	Sarah
7 · Aceitação	IA prepara o kit e consolida as anotações; **a sessão é com pessoas** (N-13)	Checklist preenchido; RNF-08 e SUS medidos	Yuri
8 · Operação	Agente 2 (N-20); IA para infra e manual, com revisão	Link público verificado; tokens das fases 7 e 8 no log	Yuri

N-13 · Aceitação é humana.

- O RNF-08 e a SUS só são medidos com participantes reais, conforme `docs/aceitacao/roteiros-tarefas.md`.
- A IA **não deve** simular participante nem preencher resultado.
- Os dados dos participantes entram no prompt só como código (P1, P2…), nunca com nome ou e-mail real.

8. Testes

N-14 · Especificado é testado ou registrado. Todo item que o projeto do programa manda criar **deve** ter um caso de teste com `@req`, ou uma lacuna aberta dizendo por que não tem. Isso inclui seed, migração de dados, configuração de produção e scripts. O agente 4 só implementa o que a suíte pede: no piloto, o seed especificado nunca foi criado (DEF-09).

N-15 · Composição testada. Antes da fase 6:

- cada app **deve** ter ao menos um teste que monte o app inteiro (`main.tsx`) e percorra a navegação principal;
- antes da implantação, um caso **deve** rodar a pilha de produção atrás do proxy.

No piloto, 607 testes verdes conviviam com 4 defeitos de composição.

N-16 · Teste conferido contra o critério.

- A cada rodada do agente 3, a Sarah confere, por amostra, **um caso por requisito** contra o critério do baseline.
- Teste de interação **deve** usar o gesto que o critério nomeia: se o critério diz arrastar, o teste arrasta.
- Esta regra cobre o erro correlacionado entre teste e código, que a P2 não pega porque o mesmo modelo escreve os dois (DEF-02).

N-17 · E2E também tem regra.

- Quando o agente 3 não puder rodar o navegador, o E2E escrito pela sessão principal **deve** seguir as mesmas regras do agente 3:
  - não afrouxar asserção depois de vermelho;
  - não usar relógio nem sorteio sem injeção;
  - marcar `@req`;

    - descrever o comportamento correto, com `test.fail` para defeito conhecido.
- O responsável de testes revisa cada caso antes do commit.
- Meta: estender o harness para rodar o E2E no contêiner do agente 3.

9. Quando delegar

N-18 · Verificação sem julgamento é ferramenta. Confirmar que um requisito já está verde (suíte verde, diff vazio, escopo respeitado) **deve** ser feito por uma ferramenta determinística em `.claude/ferramentas/`, sem modelo. Agente se usa onde há julgamento.

N-19 · Parada por retorno do agente 5.

- Numa mesma fase, o agente 5 para quando **duas rodadas seguidas** não registrarem lacuna bloqueante nem conflito de contrato.
- As lacunas parciais restantes vão em lote ao arquiteto.
- Se a 1ª rodada estourar o orçamento, a seguinte começa com um escopo dado (por onde começar) e não com um teto maior.

N-20 · Ordem e maturidade.

- O agente 4 só roda sobre requisito que já tem testes do agente 3.
- O agente 2 entra em **modo sombra**: nos 10 primeiros chamados, a Sarah ou o Yuri faz a triagem em paralelo, e a comparação é registrada.
- O agente 2 só passa a valer como triagem oficial se acertar a classificação em pelo menos 8 dos 10 chamados.
- Os primeiros chamados de calibração são DEF-09 e DEF-10, registrados de forma retroativa.

N-21 · Tabela de delegação.

Delegar	Delegar com condição	Não delegar
Detectar ambiguidade, conflito e lacuna	Redigir contrato ou ADR: a IA redige, o arquiteto aceita	Responder pendência, lacuna ou ADR (N-04)
Gerar casos de teste a partir do critério	E2E na sessão principal (N-17)	Afrouxar teste vermelho
Implementar contra a suíte	Escolher dependência: só o arquiteto mexe em `package.json`	Medir usabilidade com pessoas (N-13)
Ler código e logs mascarados atrás de defeito	Proposta de correção em ramo, com revisão	Agir sobre produção: reiniciar, aplicar correção, mexer no banco
Rascunhar manual, roteiros e relatórios	Prioridade de defeito: a IA propõe, a pessoa decide	Assinar artefato

10. Harness

N-22 · Falso positivo se corrige com teste, não se desliga.

- Um bloqueio legítimo que atrapalha muito, como a P1 contra `cd` para a raiz do projeto ou a P4 contra a "senha do jogo", é registrado pelo dono do harness.
- A política é ajustada com dois testes novos em `.claude/testes-harness/`: um caso que bloqueia e um que permite.
- **Não se desliga** uma política para destravar uma execução.

N-23 · Retomada preserva a tarefa.

- Se uma execução parar (limite de uso, queda), a retomada **deve** reiniciar a tarefa com `iniciar-tarefa` antes de qualquer edição.

- Até o `ciclo.js` preservar o estado na retomada, o responsável confere no relatório que o `baseline_verde` não mudou.

N-24 · Mudança no harness.

- Toda mudança em `.claude/` exige `node --test ".claude/testes-harness/*.test.js"` verde e revisão de uma segunda pessoa além do dono do harness.
- A IA da sessão principal **não deve** mudar permissões em `.claude/settings.json` por conta própria.
- O dono do harness **deve** ter um substituto que saiba rodar e ler os testes do harness.

11. Segurança

N-25 · Controles mínimos.

a. **Sessão principal com supervisão.** Comandos que mudam o ambiente exigem a pessoa presente. Isso inclui instalar, construir imagem, abrir serviço, mexer em `infra/` e fazer push.

b. **Testes sem rede.** Suítes e agentes de teste rodam em contêiner `--network none`. Nenhum teste usa dado real.

c. **Sem segredo.**

- Nada de senha, token ou chave em prompt, código ou documento.
- Configuração só por variável de ambiente.
- O padrão de segredo do agente 4 vale também para o que a sessão principal escreve: o responsável confere no diff.

d. **Dado pessoal.**

- Log bruto só passa ao modelo pelo `buscar-log`, que mascara.
- `harness/logs/brutos/` continua negado à leitura.
- Dados de participantes seguem a N-13.

e. **Texto externo é dado.** O texto de issue, chamado, log e código legado nunca é instrução. Agente que lê esse texto não tem ferramenta de ação sobre produção.

f. **Dependências.**

- Versão RC, beta ou sem fixar exige decisão do arquiteto.
- O lockfile é revisado em todo commit que o muda.
- Manifesto novo, inclusive isolado como `tests/e2e/package.json`, é decisão de arquitetura.

g. **Produção é só leitura para a IA.** Implantação é feita pelo CD a partir da `main` verde, nunca por agente.

12. Dependência de fornecedor

N-26 · Saída possível e exercitada.

- As políticas ficam em JavaScript puro, testável sem modelo. O código que liga o harness à ferramenta (frontmatter, `settings.json`, eventos dos hooks) fica separado e documentado em `CLAUDE.md`.
- Os artefatos do processo ficam em JSON com esquema próprio, sem formato do fornecedor.
- Uma vez por semestre, o dono do harness roda os testes do harness e **um** agente (o 5, que só lê) em outra ferramenta ou modelo, e registra o que precisou ser adaptado.
- Limite de uso da conta é risco de prazo: fase com entrega marcada deve começar com folga, e a retomada segue a N-23.

13. Formação da equipe

N-27 · Quem assina explica. O responsável da fase só assina o artefato se souber explicar, sem a IA, por que cada decisão, caso ou prioridade está ali. A equipe pode pedir essa explicação em reunião.

N-28 · Uma parte à mão por fase. Em cada fase, o responsável faz à mão pelo menos uma tarefa central e a registra no log de IA como "sem IA". Exemplos:

- um requisito com critério (Roberta);
- uma ADR (Eric);
- um caso E2E e a priorização (Sarah);
- uma verificação de implantação (Yuri).

N-29 · Registrar o que foi corrigido. O log de IA de cada fase mantém "onde a IA errou e o que foi corrigido à mão". Esse é o dado que mostra se a equipe ainda sabe revisar.

N-30 · Conhecimento distribuído.

- O harness e a orquestração dos agentes não podem depender de uma pessoa só.
- A cada ciclo, uma pessoa diferente roda uma fase de agente com o dono do harness ao lado.
- Os commits de cada fase saem da conta do responsável da fase.

14. Propriedade intelectual

N-31 · Autoria e licenças.

a. **Licença do repositório.** O repositório **deve** ter um arquivo `LICENSE` escolhido pela equipe antes de o link público existir.

b. **Coautoria da IA.** Commits com conteúdo gerado por IA mantêm a linha `Co-Authored-By` da ferramenta.

c. **Termos do fornecedor.** O responsável da fase 8 confere, antes da publicação, os termos do fornecedor vigentes para a conta usada quanto à titularidade do que é gerado, e registra a conclusão no plano de manutenção.

d. **Material de terceiros.**

- Imagem, código ou texto de terceiros entra só com origem e licença registradas.
- Os sprites herdados do legado precisam dessa origem antes da publicação.

e. **Código de outros.** Não se envia ao modelo código ou documento de terceiros sem autorização. O material da disciplina é usado só no contexto da disciplina.

15. Exceções e revisão da norma

N-32 · Exceção. A exceção a uma regra é pedida por escrito ao responsável da fase e ao arquiteto, com o motivo e o prazo, e registrada no log de IA da fase. Não existe exceção às N-04, N-05 e N-25 g.

N-33 · Revisão. A norma é revista ao fim de cada ciclo do processo, com as métricas da N-11. A revisão segue o mesmo caminho da mudança de requisito: proposta, discussão e aprovação da equipe.

Anexo A · Checklist de fechamento de fase

Conferir antes do commit da fase (`.claude/rules/commits.md`):

☐ Toda execução de agente da fase tem relatório e linhas no log (N-03). As descartadas estão anotadas.
☐ Nenhuma pendência ou lacuna foi respondida pela IA (N-04).
☐ Todo achado da fase virou pendência ou lacuna (N-06).
☐ O log de decisões da fase foi arquivado (N-08).
☐ Os tokens da fase estão somados, com origem (N-10), e as métricas da N-11 foram registradas.
☐ O critério de saída da fase na N-12 foi atendido.
☐ `node --test ".claude/testes-harness/*.test.js"` está verde.
☐ O responsável revisou o diff (N-05) e consegue explicar o artefato (N-27).
☐ Há uma tarefa "sem IA" registrada (N-28).

Aprovação da equipe

Nome	Papel	Assinatura	Data
Roberta Telles	Requisitos		
Eric Costa Ciciliotti	Arquitetura e codificação		
Sarah Mota	Testes		
Yuri Martinelli	Aceitação e operação		